Übung 3

Responsive Web Design

Einführung in Webengineering · WS 2026/27

In fünf Aufgaben passen Sie eine Webseite an verschiedene Bildschirmbreiten an. Sie arbeiten mit Media Queries, flexiblen Größen, Flexbox, Grid und responsiven Bildern.

01 Viewport und Media Queries verstehen

a) Unterscheiden Sie eine fließende Größenänderung von einem Layoutwechsel an einem Breakpoint. Erklären Sie Mobile First und den Einsatz von Media Queries mit `min-width`.

b) Kopieren Sie Ihre Lerngruppen-Webseite aus Übung 2 nach `kapitel03`. Verwenden Sie für `index.html` und `material.html` weiterhin eine gemeinsame Datei `css/styles.css`. Ergänzen Sie im `head` beider Seiten die folgende Angabe und erklären Sie ihre beiden Werte:

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

c) Für einen Absatz mit der Klasse `hinweis` gelten nur die folgenden Regeln. Bestimmen Sie Text- und Hintergrundfarbe bei `375px`, `600px` und `1000px` Viewportbreite. Warum greifen bei genau `600px` beide Media Queries?

```
.hinweis { color: gray; background-color: white; }
@media (min-width: 600px) { .hinweis { color: royalblue; } }
@media (max-width: 600px) { .hinweis { background-color: #fff4cc; } }
```

02 Flexible Größen und gemeinsame Werte verwenden

Ein `div` der Klasse `seite` ist direktes Kind von `body` und enthält eine `h1`. Für die Berechnung gibt es keine Innenabstände, Rahmen oder weiteren Größenregeln. Der `body` füllt die Viewportbreite:

```
html { font-size: 16px; }
:root { --rand: 1rem; --akzent: #154a6c; }
body { margin: 0; }
.seite {
  width: calc(100% - 2 * var(--rand));
  max-width: 60rem;
  margin: 0 auto;
}
h1 { font-size: calc(1rem + 2vw); color: var(--akzent); }
```

a) Berechnen Sie bei `375px` und `1200px` Viewportbreite die Breite von `.seite`, den Abstand auf jeder Seite und die Schriftgröße der `h1`. Worauf beziehen sich `%`, `rem` und `vw` hier?

b) Übernehmen Sie die Regeln in Ihre Webseite und prüfen Sie die Werte im Browser. Ändern Sie anschließend nur `--akzent`. Erläutern Sie den Zweck von `:root` und `var()`.

Navigation und Seitenlayout anpassen

03 Mit Flexbox ausrichten und Platz verteilen

a) Gestalten Sie die Navigation mit Flexbox: unter `600px` stehen die Links untereinander, ab `600px` nebeneinander und rechtsbündig. Bei Platzmangel dürfen sie umbrechen. Verwenden Sie `gap: 0.5rem` und geben Sie an, welches Element der Flex-Container ist, welche Elemente die Flex-Items sind und wie die Hauptachse in beiden Ansichten verläuft.

b) Ein separater Testcontainer enthält drei Karten A, B und C. Es gibt keine Abstände, Rahmen oder weiteren Größenregeln und keinen Zeilenumbruch. Es gelten:

```
.karten { display: flex; width: 750px; }
.karte { flex: 1 1 150px; min-width: 0; }
```

Erklären Sie die drei Werte von `flex`. Berechnen und prüfen Sie die Kartenbreiten bei `750px` und `360px` Containerbreite. Was passiert bei `360px`, wenn alle Karten `flex-shrink: 0` erhalten? `min-width: 0` erlaubt das Schrumpfen unter die inhaltsabhängige Mindestbreite.

04 Mit Grid ein responsives Seitenraster erstellen

a) Erstellen Sie im neuen Ordner `aufgabe04` eine `index.html` mit Viewport-Metatag und eine verknüpfte `styles.css`. Verwenden Sie im `body` diese eigenständige Struktur:

```
<div class="layout">
  <header><h1>Lerngruppe Webengineering</h1></header>
  <nav><a href="#inhalt">Zum Inhalt</a></nav>
  <main id="inhalt"><h2>Inhalt</h2><p>Wir lernen HTML und CSS.</p></main>
  <aside><h2>Termine</h2><p>Mittwoch, 14 Uhr</p></aside>
  <footer>Lerngruppe Webengineering</footer>
</div>
```

b) Gestalten Sie `.layout` mit CSS Grid und `gap: 16px`. Unter `900px` Viewportbreite stehen alle fünf Bereiche untereinander. Ab `900px` stehen `main` und `aside` in zwei Spalten (`2fr 1fr`); `header`, `nav` und `footer` reichen jeweils über beide Spalten. Verwenden Sie `grid-template-areas` und `grid-area`.

c) In einer breiten Ansicht hat der Grid-Container `916px` Inhaltsbreite. Berechnen Sie bei `16px` Spaltenabstand beide Spaltenbreiten. Zusätzliche Mindestbreiten greifen nicht. Was bedeutet `fr`, und weshalb eignet sich Grid für dieses Seitenraster?

05 Bilder anpassen und das Ergebnis testen

a) Binden Sie in Ihrer Webseite aus `kapitel03` im `main` mit `picture` zwei lokale Bildvarianten ein: bis einschließlich `600px` eine schmale, darüber eine breite Variante. Verwenden Sie `source`, `srcset`, `media` und ein `img` mit passendem Alternativtext. Das Bild soll in seinen Container passen, sein Seitenverhältnis behalten und nicht über seine natürliche Breite hinaus wachsen.

b) Testen Sie beide Seiten Ihrer Webseite und das separate Grid-Beispiel bei `375px`, `768px` und `1200px` sowie direkt unter, auf und über dem jeweiligen Breakpoint (`600px` bzw. `900px`). Prüfen Sie auch `200%` Browserzoom. Beheben Sie Überlappungen, abgeschnittene Inhalte und unnötiges horizontales Scrollen. Notieren Sie drei Beobachtungen mit der verantwortlichen HTML- oder CSS-Regel.